HWA CHONG INSTITUTION
C2 TIMED PRACTICE EXERCISE 2024

COMPUTING
Higher 2

20 May 2024 **Paper 2 (9569 / 02)** **0815 – 1015 hrs**

Additional Materials:

Electronic version of `BOOKS.txt` data file

Electronic version of `CLASS.txt` data file

Electronic version of `ITEMS.txt` data file

Electronic version of `PROGRAM.json` data file

Insert Quick Reference Guide

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

The maximum mark for this paper is **65**.

The number of marks is given in brackets [ ] at the end of each question or part question.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Instructions to candidates:

Your program code and output for each of Task 1, 3 and 4 should be saved in a single `.ipynb` file using Jupyter Notebook. For example, your program code and output for Task 1 should be saved as:

`TASK1_<your name>_<CT>.ipynb`

Make sure that each of your `.ipynb` files shows the required output in Jupyter Notebook.

This document consists of **7** printed pages.

1 Name your Jupyter Notebook as `TASK1_<your name>_<CT>.ipynb`.

The task is to create a NoSQL database for an online platform to manage the programs (movies or TV series) for its customers. Each program contains the title, year, genre and rating. It may include optional information, e.g. top cast, the season number and the total number of episodes in the season.

For each of the sub-task, add a comment statement at the beginning of the code using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [1]:  # Task 1.1
         Program Code
         output:
```

Task 1.1

Write program code to:

- Create a MongoDB database named `Entertainment` and a collection named `Program`.
- Use the dataset in the file `PROGRAM.json` to insert the documents in the `Program` collection in the `Entertainment` database.
- Display all the documents in the `Program` collection. [3]

Task 1.2

Write program code, which makes use of the `Program` collection in the `Entertainment` database to display the following information:

- Programs without a top cast.
- Programs with rating above 7 and genre of either Sci-Fi or Mystery.
- Number of programs with top cast Taylor Swift.
- Total number of episodes in Black Mirror.

All output should have appropriate messages to indicate what you are showing. [6]

Task 1.3

Write program code to remove all the movies titled 'John Wick'. Display all the documents after the removal. [2]

Save your Jupyter Notebook for Task 1.

2 The task is to implement a client program and a server program using socket programming in Python.

The client program accepts input from the user and search for the item. The server program will manage a collection of items stored in a suitable data structure.

Task 2.1

Write code to implement a client program that:

- Prompt the user for an article number or "`exit`" to terminate the program and close the connection.
- Send the article number to the server.
- Display article name received from the server.
- Display "`Not Found`" message, if there are no matching article number found.
- Send "`bye`" to the server when user input "`exit`".

Save your program code as `TASK2_Client_<your name>_<CT>.py`. [4]

Task 2.2

The text file `ITEMS.txt` stores the comma-separated values for a unique `ArticleNo` and `Name`.

Write code to implement a server program that:

- Read in the data from the file `ITEMS.txt` and manage a collection of items.
- Search for an item in the collection upon receiving the article number from the client.
- Send the article name back to the client if it is found, otherwise send "`Not Found`".
- Close and terminate the program upon receiving "`bye`" from client.

Save your program code as `TASK2_Server_<your name>_<CT>.py`. [7]

3 Name your Jupyter Notebook as `TASK3_<your name>_<CT>.ipynb`.

All students in a primary one class were born in the year 2001. A binary search tree is used to store the name and date of birth for the class. The tree is arranged in order of the student's age, with older students appearing on the left and younger students on the right. If two students have the same date of birth, they are arranged in ascending order of their names.

The tree is implemented using Object-Oriented Programming (OOP).

The class `TreeNode` contains four attributes:

- `name` the student's name
- `DOB` the student's date of birth in the form of YYYYMMDD. For example, '20010123' for student born on Jan 23, 2001
- `left` points to the left node
- `right` points to the right node.

The class `TreeNode` contains one method:

- A constructor to set `left` pointer and `right` pointer to `None`, `name` and `DOB` to their parameters.

The class `Tree` contains one attribute:

- `root` points to the root node `TreeNode` in the tree.

The class `Tree` contains the following methods:

- A constructor to create an empty tree.
- A method to take the parameters and store them as a `TreeNode` in the correct position in the tree.
- A recursive method to use in-order traversal to output the `name` and `DOB` of each student in the tree.
- A recursive method to take a parameter `month` and return the total number of students born in this month. The parameter `month` is formatted as a string, e.g. '01' means January.

For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [1]: # Task 3.1
        Program Code
        output:
```

Task 3.1

Write program code to declare the classes `TreeNode` and `Tree`, and their constructors. [2]

Write program code to:

- Declare the method to insert a new node into the tree.
- Declare the recursive method to output the in-order traversal of the tree.
- Declare the recursive method to return the total number of students born in the month. [13]

Task 3.2

Write program code to:

- Read in the data in the file `CLASS.txt`
- Declare a new instance of `Tree` and store each student's name and DOB as a `TreeNode` in the tree.
- Call the in-order method to show the output of in-order traversal.
- Display the total number of students born in each month, as shown in the sample output below.

Month	Total
01	5
02	0
03	1
04	1
05	1
06	0
07	1
08	0
09	0
10	1
11	0
12	0

[6]

Save your Jupyter Notebook for Task 3.

4 Name your Jupyter Notebook as `TASK4_<your name>_<CT>.ipynb`.

The task is to create a program that allow user to manage an **array** of book objects and search for the location of the book in the library.

Python List's methods and operators should not be used in the implementation of the searching and sorting algorithm except for array creation. Example: `append, pop, +, slicing` etc.

For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [1]:  #Task 4.1
         Program Code
         output:
```

Task 4.1

The class `Book` contains two properties:

- `title`
- `location`

Write program code to declare the class `Book` and its constructor. [1]

Task 4.2

One method of sorting is the quicksort.

Write a function `task4_2(array, left, right)` that takes in 3 arguments and sort the `array` in ascending order of the book's title using the quicksort algorithm.

- `array` – the array of `Book` objects
- `left` – left index of the array to sort
- `right` – right index of the array to sort. [6]

Task 4.3

One method of searching is the binary search.

Write a function `task4_3(array, left, right, title)` that takes in 4 arguments and search for the `title` in the `array` using the binary search algorithm. The function returns the `Book` object found or `None` if not found.

- `array` – the sorted array of `Book` objects
- `left` – left index of the array to search
- `right` – right index of the array to search
- `title` – title of the book to search. [4]

Task 4.4

The text file `BOOKS.txt` stores the comma-separated values for book title and location.

Using the functions in Task 4.2 and Task 4.3, write program code to:

- Create a fixed size array of 50 elements. Name the array `bookArr`.
- Read the content in file `books.txt` into array `bookArr`.
- Sort array `bookArr` using function `task4_2`.
- Accept user input to search for a book by title using function `task4_3`.
- Display the book's information as the result if it is found, otherwise, display `"Not Found"`.

Test your program using the titles `"Effective Pythonic Patterns"` and `"Dart Game Development"`, and show the output. [5]

Task 4.5

In the insertion sort algorithm, every pass moves an element from the unsorted portion to the sorted portion of the array until all the elements are sorted in the array.

Write program code to

- Accepts user input to add one or more books into the sorted array `bookArr`.
- Use a modified insertion sort algorithm to maintain the sorted order of `bookArr` with minimum number of passes after all new books are added.

Test your program by adding `"C# for Web Development,Yellow Zone Shelf 14"` and `"Effective C++ Paradigms,Green Zone Shelf 21"` to `bookArr` in Task 4.4.

Display the books' information in `bookArr` after adding. [6]

Save your Jupyter Notebook for Task 4.